

The identification of network interfaces in Linux and Unix-like systems has evolved from a simple kernel-assignment model to a complex, rule-based "predictable" naming convention. In the early days of Linux, the kernel assigned names like `eth0` or `wlan0` based on the order in which drivers were initialized during the boot process [Nemeth, Evi, et al]. However, as hardware became more dynamic—with the introduction of hot-pluggable USB network adapters and multi-port PCIe cards—this method became unreliable because the order of initialization could change between reboots, leading to "interface swapping" [Ward, Brian]. Modern systems primarily utilize `systemd` and `udev` to assign persistent names based on physical hardware location, MAC addresses, or firmware-provided indices [Shotts, William].

Standard Command-Line Utilities

The most direct method to find available network interfaces on a modern Linux system is using the `ip` command from the `iproute2` package, which has largely superseded the legacy `net-tools` (`ifconfig`) [Rosen, Kenneth, et al].

- **`ip link show`**: This command lists all network interfaces recognized by the kernel, including those that are currently "down" or inactive. It provides the interface name, its current state (UP/DOWN), and its MAC address [`ip-link(8) man`].
- **`ip addr`**: This provides the same list as `ip link` but includes assigned IP addresses (IPv4 and IPv6) [`man`].
- **`ifconfig -a`**: While deprecated in many modern distributions, the `-a` flag is essential because a standard `ifconfig` call only displays active (UP) interfaces. The `-a` flag forces the display of every interface the kernel has detected [`ifconfig(8) man`].
- **`nmcli device`**: For systems running NetworkManager, this tool provides a high-level view of interfaces, their types (ethernet, wifi, loopback), and their connection status [`nmcli(1)`].

The Filesystem Approach: `/sys` and `/proc`

In Linux, the kernel exposes hardware information through virtual filesystems. This is often the most "script-friendly" way to list interfaces without parsing complex command output [Love, Robert].

- **`/sys/class/net/`**: Every subdirectory within this path represents a network interface. A simple `ls /sys/class/net` will return a clean list of interface names (e.g., `lo`, `enp3s0`, `wlo1`) [The `/sys` Filesystem]. This directory is preferred over `/proc` because it contains detailed subdirectories for each device, such as `/sys/class/net/<iface>/address` for the MAC address and `/sys/class/net/<iface>/operstate` for the operational status [Negus, Christopher].
- **`/proc/net/dev`**: This file provides a summary of all interfaces along with receive and transmit statistics. It is useful for verifying which interfaces are actually seeing traffic [`proc(5)`].

Predictable Network Interface Naming

The "Predictable Network Interface Names" scheme, implemented by `systemd-udev`, uses several strategies to name devices so they remain constant even if hardware is added or removed [Shotts, William + Debian Wiki]. The naming hierarchy generally follows this priority:

1. **Onboard (eno1)**: Names derived from firmware/BIOS indices for onboard devices.
2. **PCI Express Hotplug (ens1)**: Names derived from the PCIe hotplug slot index.
3. **Physical Location (enp2s0)**: Names derived from the hardware's physical connector location (Bus 2, Slot 0).
4. **MAC Address (enx78e7d1ea46da)**: Used primarily for USB devices where physical location is not fixed [Debian Wiki + Freedesktop.org].

To predict what a name *will* be or to debug naming issues, the command `udevadm test-builtin net_setup_link /sys/class/net/<device>` can be used. This utility queries the `udev` database to show which naming policy is being applied to a specific hardware path [Debian Wiki].

Programmatic Identification (C/C++)

For developers needing to find interfaces within software, the standard Unix method is the `getifaddrs()` function [Stevens, Richard + Stephen Rago]. This function creates a linked list of `ifaddrs` structures, each representing a network interface and its associated addresses.

- **getifaddrs()**: Returns 0 on success and populates a pointer to the list. Each entry contains the `ifa_name` (the string name of the interface) and `ifa_flags` (to check if the interface is UP, LOOPBACK, etc.) [getifaddrs(3)].
- **Netlink Sockets**: On Linux specifically, the Netlink protocol (`AF_NETLINK`) allows programs to communicate with the kernel to request link information (`RTM_GETLINK`). This is more complex but provides the most detailed real-time information about the network stack [rtnetlink(7)].

Hardware Discovery Tools

If an interface does not appear in `ip link` or `/sys/class/net`, it usually indicates a missing kernel driver or a hardware failure. Tools to verify the physical presence of the NIC include:

- **lspci -knn**: Lists all PCI devices and specifically shows which kernel driver (module) is currently handling the network controller [lspci(8)].
- **lshw -C network**: Provides a comprehensive summary of network hardware, including logical names, serial numbers, and capabilities (e.g., 1000Mbit/s) [lshw(1)].

Sources

1. Nemeth, Evi, et al. *UNIX and Linux System Administration Handbook*. 5th ed., Pearson Education, 2017. (Print)
2. Ward, Brian. *How Linux Works: What Every Superuser Should Know*. 3rd ed., No Starch Press, 2021. (Print)
3. Shotts, William. *The Linux Command Line: A Complete Introduction*. 2nd ed., No Starch Press, 2019. (Print)
4. Rosen, Kenneth, et al. *UNIX: The Complete Reference*. 2nd ed., McGraw-Hill Education, 2007. (Print)
5. "ip-link(8) - Linux manual page." man7.org

6. "ip-address(8) - Linux manual page." man7.org
7. "ifconfig(8) - Linux manual page." man7.org
8. "nmcli(1) - Linux manual page." man7.org
9. Love, Robert. *Linux Kernel Development*. 3rd ed., Addison-Wesley Professional, 2010. (Print)
10. "The /sys Filesystem." kernel.org
11. Negus, Christopher. *Linux Bible*. 10th ed., Wiley, 2020. (Print)
12. "proc(5) - Linux manual page." man7.org
13. Debian Wiki. [NetworkInterfaceNames](https://wiki.debian.org/NetworkInterfaceNames)
14. Freedesktop.org. [Predictable Network Interface Names](https://freedesktop.org/PredictableNetworkInterfaceNames)
15. Stevens, Richard, and Stephen Rago. *Advanced Programming in the UNIX Environment*. 3rd ed., Addison-Wesley Professional, 2013. (Print)
16. "getifaddrs(3) - Linux manual page." man7.org
17. "rtnetlink(7) - Linux manual page." man7.org
18. "lspci(8) - Linux manual page." man7.org
19. "lshw(1) - Linux manual page." die.net
20. Stack Overflow. [C code to get the interface name for the IP address in linux](https://stackoverflow.com/questions/4014744/c-code-to-get-the-interface-name-for-the-ip-address-in-linux)
21. Unix StackExchange. [How can I find available network interfaces](https://unix.stackexchange.com/questions/111111/how-can-i-find-available-network-interfaces)

San Anton [20260221 via AHR “PNI? we talked about prior 2019 at gihub and of course is nothing new but a jargon meaningful enough to symbolize transition from the old NIC”]

PS: Have no idea what date back-then ... but this ls is making sense even today: `pni_if=$(ls /sys/class/net)` – “the start small llc” makes me laugh. If is not there, one needs a big library with all manufacturers in the world. sananton.substack...